

The Command Line and Git Guide for PWTC Students

Created by Gianna Leidich, Teaching Assistant for the Spring 2026 semester of English-391C.

Introduction

In English-391C, you learn about file management and GitHub, but only ever by using graphical user interfaces: File Explorer, if you are on Windows OS, or Finder, if you are on MacOS, and the GitHub website, respectively.

This guide advances your knowledge of both file management and GitHub a step further by introducing you to the **command line interface**, a text-based interface that allows you to interact with your file system and Git repository using written commands.

In this guide, you will find two sections:

- An overview of the command line and basic commands.
- A tutorial on creating, cloning, and interacting with your local and remote repositories via the command line in VSCode.

Both sections are designed to be followed **step-by-step**.

To understand this guide fully, it is recommended that you have basic knowledge of file management and file paths and have already created a GitHub account.

Video tutorials

- [Clone your remote GitHub repository](#): A short tutorial on cloning an existing GitHub repository. Suggested for readers who want to clone their existing remote repository for their website, onto their local machine.
- [Git workflow example](#): A short tutorial going through the typical Git workflow including change, add, commit, and push.

Note: You must sign into SharePoint with your UMass account to access the videos.

Table of Contents

Introduction.....	1
Video tutorials	1
Basic commands	3
What is the command line?	3
Step 1: Open the terminal.....	3
Step 2: Use ls to view files and directories	3
Step 3: Use mkdir to make a new directory	4
Step 4: Use cd to enter a directory	4
Step 5: Use touch to create a new file	5
Step 5.5: Use vim to write in a file (optional)	5
Step 6: Use mv to move a file	6
Step 7: Use rm to remove a file and directory	8
Git-specific commands	9
What is Git and GitHub?	9
Create and connect a new repository	9
Step 1: Install Git	9
Step 2: Create your remote repository	9
Step 3: Create your local repository	10
Step 4: Connect your local and remote repositories	11
Step 5: Open your local repository in VSCode	13
Clone and connect an existing repository	15
Step 1: Install Git	15
Step 2: Clone your remote repository	15
Step 3: Open your local repository in VSCode	16
Review.....	17
Sources	17
Appendix	18
Basic commands cheat sheet	18
Command	18
What it does	18
Git workflow quick guide	19

Basic commands

What is the command line?

The **command line interface**, also known as simply the **command line** or as the **shell**, allows you to interact with your operating system using written, text-based commands. Anything that you can do in a graphical user interface, you can do on the command line.

Example: You typically use File Directory or Finder, both graphical user interfaces, for file management. But anything you can do in File Directory or Finder, including creating, moving, and deleting files, you can also do from the command line.

This part of the guide provides you with a brief introduction to file management via the command line.

Step 1: Open the terminal

To access the command line, we need to first open a terminal. When you start a terminal, it opens at your home directory, and this is where you are located within your file system.

MacOS: Open **Finder > Utilities > Terminal**.

Windows OS: Type **Terminal** in the **Search** textbox on the taskbar, then click **Open**.

Note: The command line and terminal are often used interchangeably. Strictly speaking, the command line is the literal line where you write the command, and the terminal is the window that displays that line.

Step 2: Use **ls** to view files and directories

1. Open a new terminal. Use this terminal for this step and all subsequent steps.
2. Type **ls** and click **Enter** to input the command. Your operating system outputs all the files and directories within your current directory in a list.

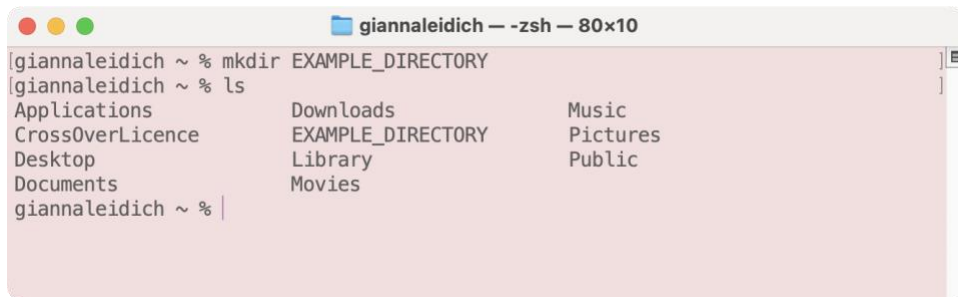


```
giannaleidich ~ % ls
Applications      Downloads          Pictures
CrossOverLicence  Library           Public
Desktop           Movies
Documents         Music
```

Fun fact: Your operating system sees everything as a file, which is why the terminal outputs both files and directories on the **ls** command.

Step 3: Use **mkdir** to make a new directory

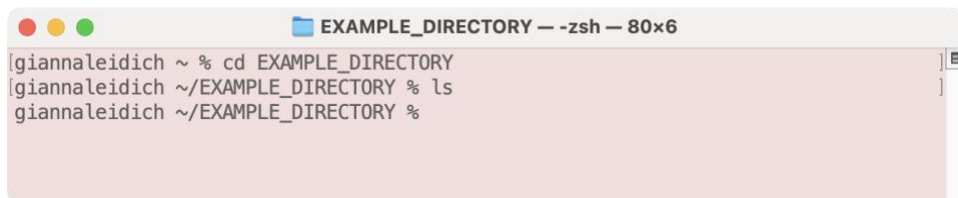
1. Input **mkdir EXAMPLE_DIRECTORY** in the same terminal. Your system creates a new directory named **EXAMPLE_DIRECTORY**, nested within your current directory.
2. Input **ls**. The output now includes **EXAMPLE_DIRECTORY**.



```
giannaleidich ~ % mkdir EXAMPLE_DIRECTORY
giannaleidich ~ % ls
Applications      Downloads          Music
CrossOverLicence  EXAMPLE_DIRECTORY Pictures
Desktop           Library           Public
Documents         Movies
```

Step 4: Use **cd** to enter a directory

1. Input **cd EXAMPLE_DIRECTORY** in the same terminal. Your system moves your location from your home directory into the example directory.
2. Input **ls**. The output does not contain any content, which is correct: our example directory does not have any nested files nor directories, so there is nothing to be output.



```
EXAMPLE_DIRECTORY ~ % cd EXAMPLE_DIRECTORY
EXAMPLE_DIRECTORY ~ % ls
EXAMPLE_DIRECTORY ~ %
```

Note: It now says **EXAMPLE_DIRECTORY** next to my username **giannaleidich** because that is my new current directory.

Step 5: Use **touch** to create a new file

1. Input **touch EXAMPLE_FILE.txt** in the same terminal. Your system creates a new file named **EXAMPLE_FILE.txt**, nested within the example directory.
2. Input **ls**. The output now includes the example file.



```
EXAMPLE_DIRECTORY — -zsh — 80x9
giannaleidich ~/EXAMPLE_DIRECTORY % touch EXAMPLE_FILE.txt
giannaleidich ~/EXAMPLE_DIRECTORY % ls
EXAMPLE_FILE.txt
giannaleidich ~/EXAMPLE_DIRECTORY %
```

Step 5.5: Use **vim** to write in a file (optional)

This step shows you how to use **vim**, a terminal-based text editor that you can run from the command line, to write some content into your example file. Using **vim** is completely optional and the rest of the steps can be followed without doing it.

1. Input **vim EXAMPLE_FILE.txt** in the same terminal. Your system opens the example file in the text editor.



```
EXAMPLE_DIRECTORY — vim EXAMPLE_FILE.txt — 80x24
~
~
~
~
~
~
~
~
```

2. Click **i** to enter Insert Mode. At the bottom of the window, **-- INSERT --** appears, indicating that you are now able to write to the file.
3. Write some content to the file. You may write whatever you want, but you must write something.



```
EXAMPLE_DIRECTORY — vim EXAMPLE_FILE.txt — 80x9
This is my example text file.
~
~
~
~
~
~
~
-- INSERT --
```

4. Click **Escape** to exit Insert Mode.
5. Type **:wq** and click **Enter**, to write your changes to the file and quit the text editor. Your system closes the text editor and returns you back to the command line.



```
EXAMPLE_DIRECTORY - vim EXAMPLE_FILE.txt - 80x9
This is my example text file.
~
~
~
~
~
~
~
:wq
```

6. Input **vim EXAMPLE_FILE.txt**. The file opens with your previous changes.

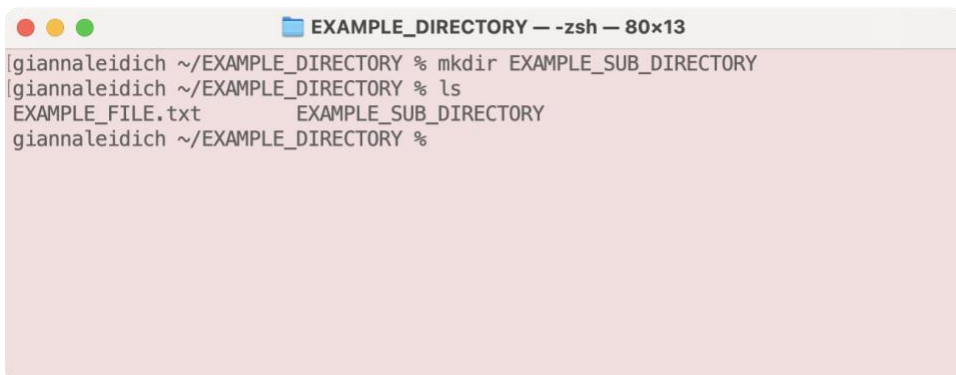


```
EXAMPLE_DIRECTORY - vim EXAMPLE_FILE.txt - 80x9
This is my example text file.
~
~
~
~
~
~
~
"EXAMPLE_FILE.txt" 1L, 30B
```

7. Click **Escape**.
8. Type **:q!** and click **Enter** to force quit the text editor. Your system closes the text editor and returns you back to the command line.

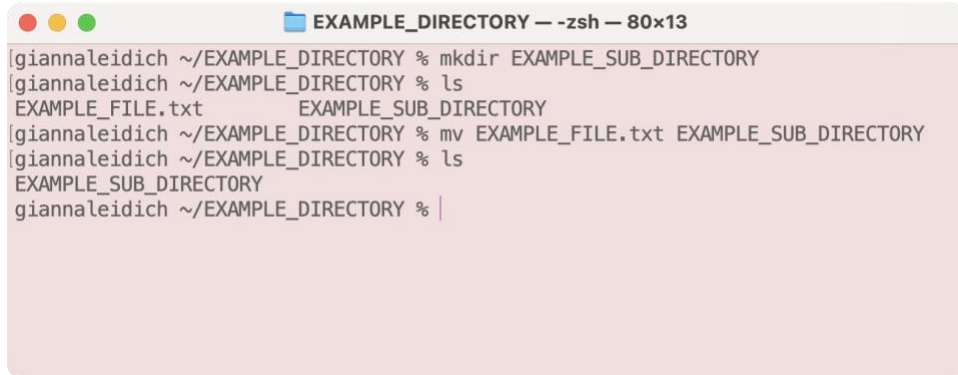
Step 6: Use **mv** to move a file

1. Input **mkdir EXAMPLE_SUB_DIRECTORY** in the same terminal. Your system creates a new directory named **EXAMPLE_SUB_DIRECTORY**, nested within your current directory.
2. Input **ls**. The output now includes the sub directory.



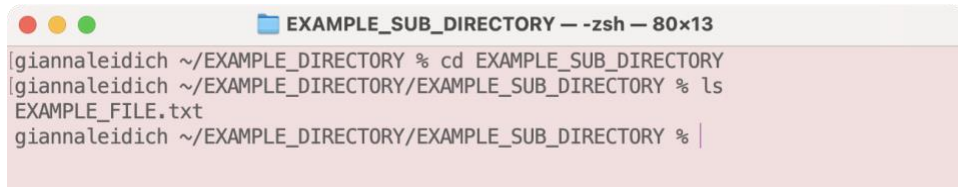
```
EXAMPLE_DIRECTORY - -zsh - 80x13
giannaleidich ~/EXAMPLE_DIRECTORY % mkdir EXAMPLE_SUB_DIRECTORY ]
giannaleidich ~/EXAMPLE_DIRECTORY % ls ]
EXAMPLE_FILE.txt          EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY %
```

3. Input **mv EXAMPLE_FILE.txt EXAMPLE_SUB_DIRECTORY**. Your system moves your example file into the sub directory.
4. Input **ls**. The output no longer includes the example file.



```
EXAMPLE_DIRECTORY — -zsh — 80x13
giannaleidich ~/EXAMPLE_DIRECTORY % mkdir EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY % ls
EXAMPLE_FILE.txt      EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY % mv EXAMPLE_FILE.txt EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY % ls
EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY %
```

5. Input **cd EXAMPLE_SUB_DIRECTORY**. Your system moves your location from the outer directory into the sub directory.
6. Input **ls**. The output for the sub directory includes the example file.

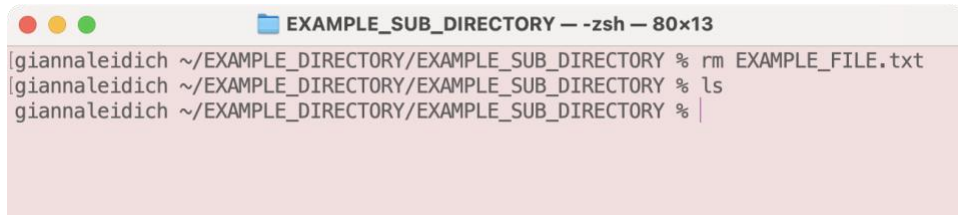


```
EXAMPLE_SUB_DIRECTORY — -zsh — 80x13
giannaleidich ~/EXAMPLE_DIRECTORY % cd EXAMPLE_SUB_DIRECTORY
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % ls
EXAMPLE_FILE.txt
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY %
```

Note: It now says **EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY** next to my username because the sub directory is now my current directory.

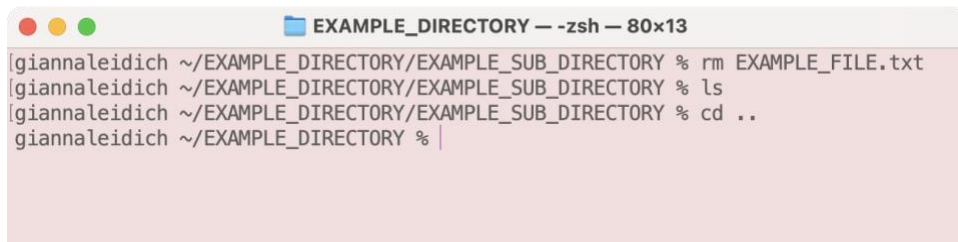
Step 7: Use **rm** to remove a file and directory

1. Input **rm EXAMPLE_FILE.txt**. Your system removes the example file from the sub directory.
2. Input **ls**. The output no longer includes the example file.



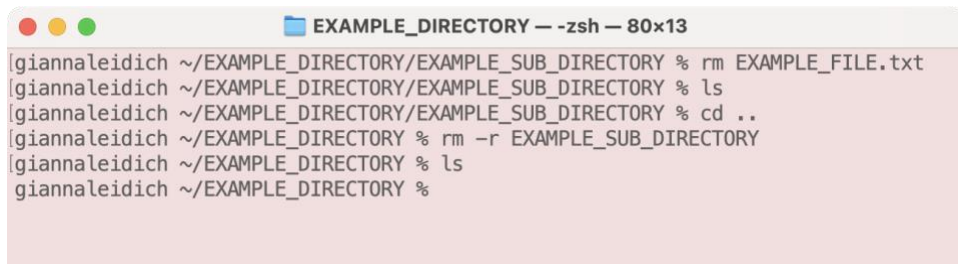
```
EXAMPLE_SUB_DIRECTORY — -zsh — 80x13
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % rm EXAMPLE_FILE.txt ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % ls ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % |
```

3. Input **cd ..**. Your system moves your location from the sub directory back into the outer directory.



```
EXAMPLE_DIRECTORY — -zsh — 80x13
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % rm EXAMPLE_FILE.txt ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % ls ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % cd .. ]
giannaleidich ~/EXAMPLE_DIRECTORY % |
```

4. Input **rm -r EXAMPLE_SUB_DIRECTORY**. Your system removes the sub directory from the outer directory.
5. Input **ls**. The output no longer contains the sub directory, or any content at all.



```
EXAMPLE_DIRECTORY — -zsh — 80x13
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % rm EXAMPLE_FILE.txt ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % ls ]
giannaleidich ~/EXAMPLE_DIRECTORY/EXAMPLE_SUB_DIRECTORY % cd .. ]
giannaleidich ~/EXAMPLE_DIRECTORY % rm -r EXAMPLE_SUB_DIRECTORY ]
giannaleidich ~/EXAMPLE_DIRECTORY % ls ]
giannaleidich ~/EXAMPLE_DIRECTORY %
```

Note: Because a directory must be empty to remove it, we needed to include the recursive flag **-r** to instruct our system to remove all sub content in the directory, if there is any, before removing the actual directory.

Git-specific commands

What is Git and GitHub?

Git is a version control system. You use Git to track all of the changes, also known as commits, that you make to your code repository, including uploading new files, making edits, and moving and deleting files.

GitHub is an online cloud-based platform that hosts and shares your Git projects. You use GitHub to store code online, view edit history, and collaborate with other users.

This part of the guide teaches you how to use Git and GitHub in combination: you use Git via the command line in VSCode to publish changes to your code from your local repository, stored on your computer, to your remote repository, hosted on GitHub.

Create and connect a **new** repository

Follow this part of the guide if you are creating a **new** repository for your website. If you already have an existing repository, skip to [Clone and connect an existing repository](#).

Step 1: Install Git

To use Git, you first need to install it onto your computer.

MacOS: First [install homebrew](#) from the official Homebrew website. Then in a new terminal, type **brew install git** and click **Enter**.

Windows OS: Follow the [Windows Git installation guide](#) from the official Git website.

Step 2: Create your remote repository

1. Open a new browser window and login to github.com.
2. Click your **profile picture** in the top-right corner. A dropdown menu opens.
3. Click **Repositories > New**.
4. Type the name for your new repository in the textbox under **Repository name**. Your repository name should be: **YOUR_USERNAME.github.io**
5. Ensure that your visibility is set to **Public**. Because we use GitHub pages for free webhosting, we must set the remote repository that stores our code to public.
6. Click **Create repository**.

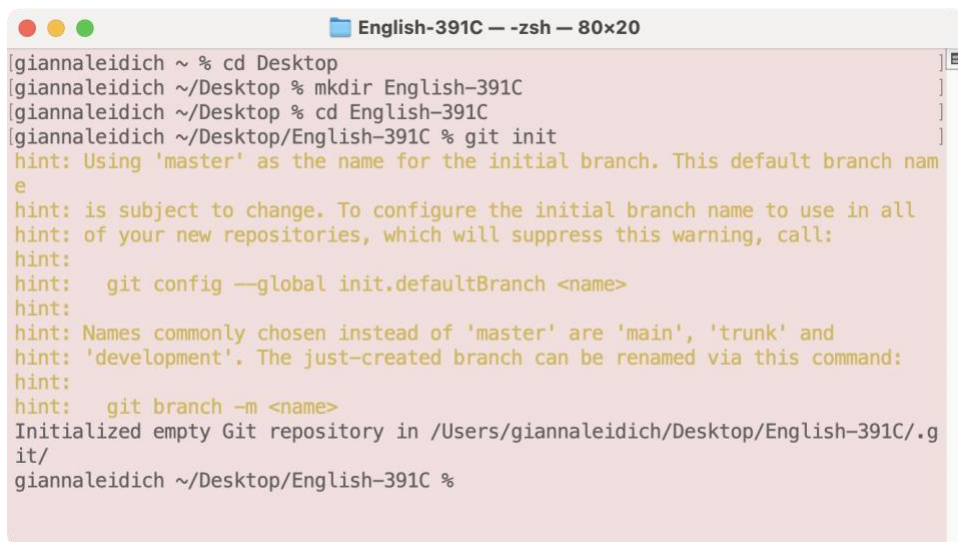
Step 3: Create your local repository

1. Open a new terminal.
2. Input **cd Desktop** to enter into your Desktop directory.
3. Input **mkdir English-391C** to create a new directory named English-391C.
4. Input **cd English-391C** to enter into the English-391C directory.



```
English-391C — zsh — 80x8
giannaleidich ~ % cd Desktop
giannaleidich ~/Desktop % mkdir English-391C
giannaleidich ~/Desktop % cd English-391C
giannaleidich ~/Desktop/English-391C %
```

5. Input **git init** to initialize your local repository.



```
English-391C — zsh — 80x20
giannaleidich ~ % cd Desktop
giannaleidich ~/Desktop % mkdir English-391C
giannaleidich ~/Desktop % cd English-391C
giannaleidich ~/Desktop/English-391C % git init
hint: Using 'master' as the name for the initial branch. This default branch nam
e
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
Initialized empty Git repository in /Users/giannaleidich/Desktop/English-391C/.g
it/
giannaleidich ~/Desktop/English-391C %
```

6. Input **touch README.md** to create an empty README file.
7. Input **git add .** to setup, also known as “stage,” the README file for saving.



```
English-391C — zsh — 80x20
giannaleidich ~/Desktop/English-391C % touch README.md
giannaleidich ~/Desktop/English-391C % git add .
giannaleidich ~/Desktop/English-391C %
```

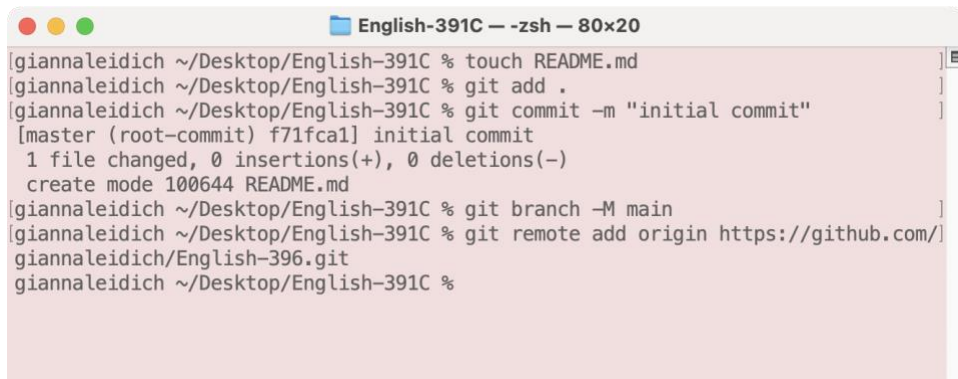
8. Input **git commit -m "initial commit"** to save, also known as “commit,” your staged changes to your local repository.



```
English-391C --zsh-- 80x20
giannaleidich ~/Desktop/English-391C % touch README.md
giannaleidich ~/Desktop/English-391C % git add .
giannaleidich ~/Desktop/English-391C % git commit -m "initial commit"
[master (root-commit) f71fca1] initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
giannaleidich ~/Desktop/English-391C %
```

Step 4: Connect your local and remote repositories

1. Input **git branch -M main** in the same terminal to set your current branch to the main branch of your repository.
2. Input **git remote add origin** https://github.com/YOUR_USERNAME/YOUR_USERNAME.github.io.git to connect your local repository to your remote repository. You can now publish, also known as “push,” any commits you make locally to your remote repository.



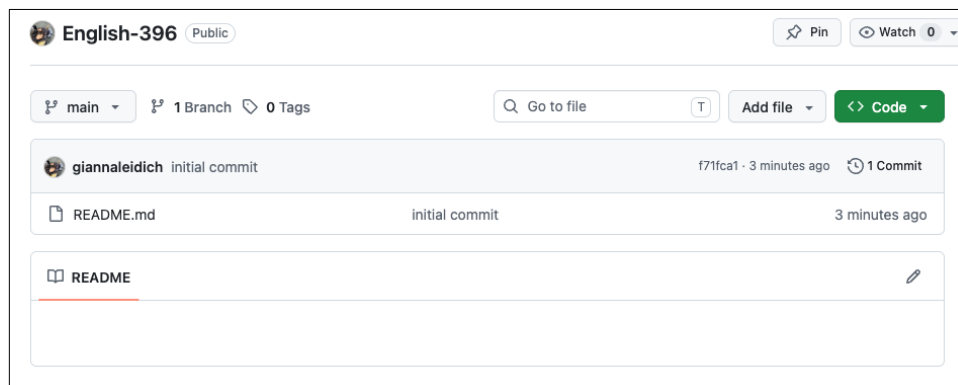
```
English-391C --zsh-- 80x20
giannaleidich ~/Desktop/English-391C % touch README.md
giannaleidich ~/Desktop/English-391C % git add .
giannaleidich ~/Desktop/English-391C % git commit -m "initial commit"
[master (root-commit) f71fca1] initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
giannaleidich ~/Desktop/English-391C % git branch -M main
giannaleidich ~/Desktop/English-391C % git remote add origin https://github.com/
giannaleidich/English-396.git
giannaleidich ~/Desktop/English-391C %
```

Note: My repository name is English-396, because I already created a repository named giannaleidich.github.io

3. Input **git push -u origin main** to push your commit from your local repository to your remote repository.

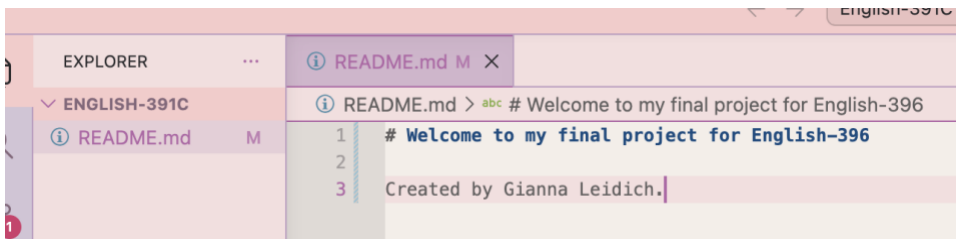
```
English-391C — -zsh — 80x20
giannaleidich ~/Desktop/English-391C % touch README.md
giannaleidich ~/Desktop/English-391C % git add .
giannaleidich ~/Desktop/English-391C % git commit -m "initial commit"
[master (root-commit) f71fca1] initial commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
giannaleidich ~/Desktop/English-391C % git branch -M main
giannaleidich ~/Desktop/English-391C % git remote add origin https://github.com/
giannaleidich/English-396.git
giannaleidich ~/Desktop/English-391C % git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 211 bytes | 211.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/giannaleidich/English-396.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
giannaleidich ~/Desktop/English-391C %
```

4. Return to your browser window and open your remote repository on GitHub. Your remote repository now includes the README file.



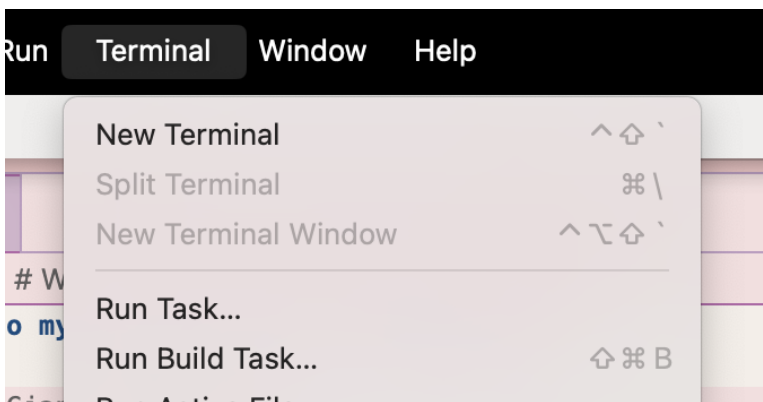
Step 5: Open your local repository in VSCode

1. Open VSCode.
2. Click the **pages icon** on the left sidebar.
3. Click **Open Folder** and open your local English-391C directory.
4. Click **README.md** and add some content, such as your name and a short description of your website.



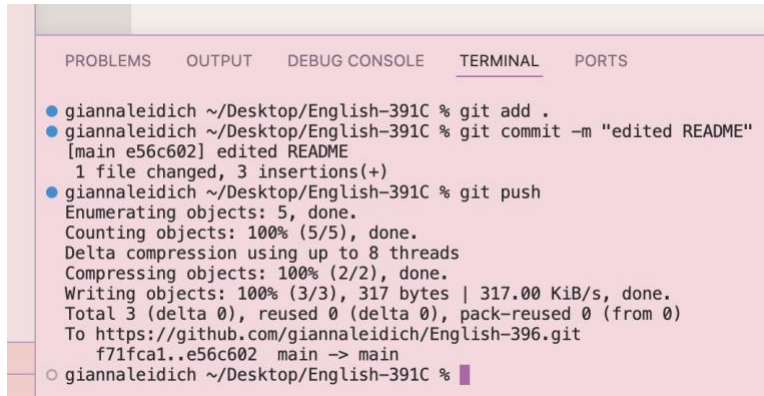
Important: Notice the shaded bar next to the lines that I just added. This bar marks your most recent changes since the last commit.

5. Click **File > Save** on the top menu bar to save your changes.
6. Click **Terminal > New Terminal** on the top menu bar. A new terminal opens at the bottom of your screen, located at your English-391C directory.



7. Input **git add .** in the terminal to stage the edits to your README file.
8. Input **git commit -m "edited README"** to commit your staged changes to your local repository.

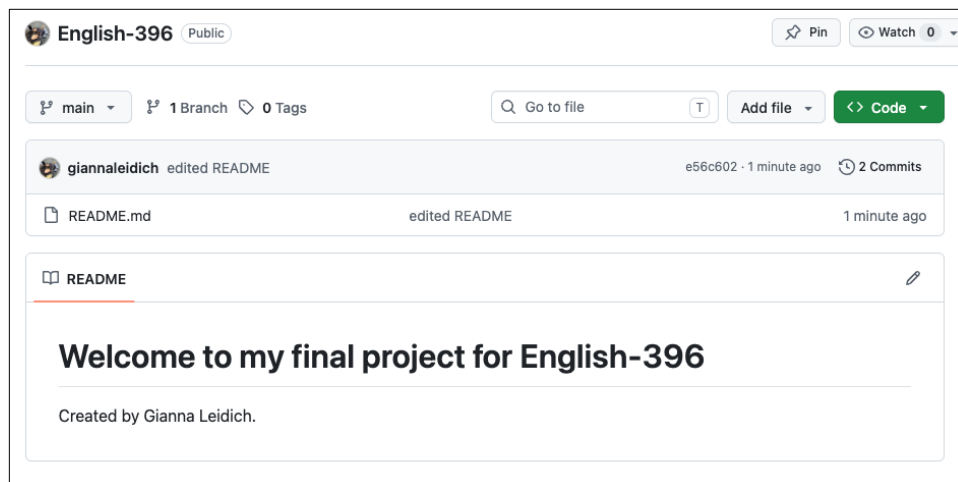
9. Run **git push** to push your commit to your remote repository.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● giannaleidich ~/Desktop/English-391C % git add .
● giannaleidich ~/Desktop/English-391C % git commit -m "edited README"
[main e56c602] edited README
1 file changed, 3 insertions(+)
● giannaleidich ~/Desktop/English-391C % git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 317 bytes | 317.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/giannaleidich/English-396.git
f71fca1..e56c602 main -> main
○ giannaleidich ~/Desktop/English-391C %
```

Important: Your workflow will always follow the same pattern of add, commit, and push, outlined in Steps 7–9.

10. Return to your browser window with GitHub opened and refresh. Your repository shows the changes you made to your README file.



Note: Because we opened a terminal located at your English-391C directory, you could use the command line and text-based commands for file management, but you might find it easier to use the VSCode interface.

Clone and connect an **existing** repository

Follow this part of the guide if you already have an **existing** repository for your website. If you need to create a new repository, jump back to [Create and connect a new repository](#).

Step 1: Install Git

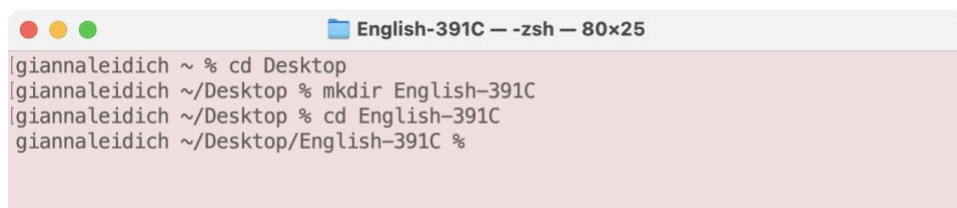
To use Git, you first need to install it onto your computer.

MacOS: First [install homebrew](#) from the official Homebrew website. Then in a new terminal, type **brew install git** and click **Enter**.

Windows OS: Follow the [Windows Git installation guide](#) from the official Git website.

Step 2: Clone your remote repository

1. Open a new terminal.
2. Input **cd Desktop** to enter into your Desktop directory.
3. Input **mkdir English-391C** to create a new directory named English-391C.
4. Input **cd English-391C** to enter into the English-391C directory.

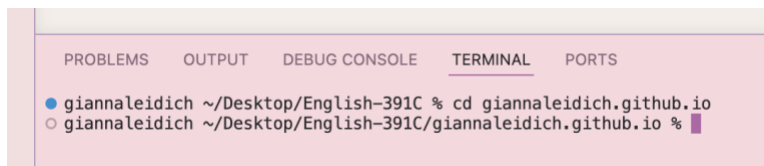
A screenshot of a macOS terminal window titled "English-391C -- zsh -- 80x25". The terminal shows a user named "giannaleidich" navigating through the file system. The commands and their outputs are: "cd Desktop" (output: "~/Desktop"), "mkdir English-391C" (output: "~/Desktop"), "cd English-391C" (output: "~/Desktop/English-391C"), and the prompt returns to "~/Desktop/English-391C %".

```
giannaleidich ~ % cd Desktop
giannaleidich ~/Desktop % mkdir English-391C
giannaleidich ~/Desktop % cd English-391C
giannaleidich ~/Desktop/English-391C %
```

5. Input **git clone**
https://github.com/YOUR_USERNAME/YOUR_USERNAME.github.io.git to create a copy, also known as “clone,” your remote repository on your local machine. This becomes your local repository.
6. Input **ls**. The output now includes a new directory named **[YOUR_USERNAME.github.io](#)**, the name of your remote repository.

Step 3: Open your local repository in VSCode

1. Open VSCode.
2. Click the **pages icon** on the left sidebar.
3. Click **Open Folder** and open your English-391C directory.
4. Click **Terminal > New Terminal** on the top menu bar. A new terminal opens at the bottom of your screen, located at your English-391C directory.
5. Input **cd YOUR_USERNAME.github.io** in the terminal. Because your local repository is nested within the **YOUR_USERNAME.github.io** directory, Git commands must be input from here, and not the outer **English-391C** folder.



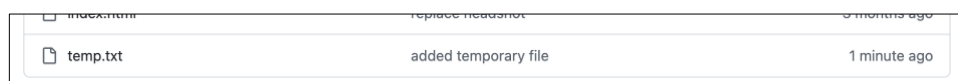
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● giannaleidich ~/Desktop/English-391C % cd giannaleidich.github.io
○ giannaleidich ~/Desktop/English-391C/giannaleidich.github.io %
```

6. Input **touch temp.txt** to create a temporary text file.
7. Input **git add .** in the terminal to stage the new file change.
8. Input **git commit -m "added temporary file"** to commit your staged changes to your local repository.
9. Run **git push** to push your commit to your remote repository.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● giannaleidich ~/Desktop/English-391C % cd giannaleidich.github.io
● giannaleidich ~/Desktop/English-391C/giannaleidich.github.io % touch temp.txt
● giannaleidich ~/Desktop/English-391C/giannaleidich.github.io % git add .
● giannaleidich ~/Desktop/English-391C/giannaleidich.github.io % git commit -m "added temporary file"
[main 7045c15] added temporary file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 temp.txt
● giannaleidich ~/Desktop/English-391C/giannaleidich.github.io % git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 273 bytes | 273.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com:giannaleidich/giannaleidich.github.io.git
e68b4a4..7045c15 main -> main
○ giannaleidich ~/Desktop/English-391C/giannaleidich.github.io %
```

10. Return to your browser window with GitHub opened and refresh. Your remote repository now includes the temporary text file.



Review

In this guide, you learned how to use the command line and text-based commands for file management and for saving and publishing changes from your local repository to your remote repository.

You can now:

- Use the command line to interact with your file system; specifically, you can create files and directories, remove and edit files, and remove files and directories.
- Create a local Git repository and connect it to a remote Git repository hosted on GitHub.
- Write changes to your local repository and push them to your remote Git repository, from the terminal in VSCode.

Sources

- The official GitHub documentation: <https://docs.github.com/en>
- The official Git documentation: <https://git-scm.com/>

Appendix

Basic commands cheat sheet

Command	What it does
ls	Lists all files in your current directory.
mkdir DIRECTORY_NAME	Makes a new directory named DIRECTORY_NAME , nested within your current directory.
cd DIRECTORY_NAME	Changes directory from your current directory. DIRECTORY_NAME must be nested within your current directory.
touch FILE_NAME	Creates a new file named FILE_NAME within your current directory.
vim FILE_NAME	Opens FILE_NAME in the vim text editor.
rm FILE_NAME	Removes the file named FILE_NAME .
rm -r DIRECTORY_NAME	Removes the directory named DIRECTORY_NAME .

Git workflow quick guide

1. Write and save changes locally in VSCode.
2. Open a terminal from the VSCode top menu bar, in the directory that contains your local repository.
 - If you created a new repository, your terminal must be located at your **English-391C** directory.
 - If you cloned an existing repository, your terminal must be located at your **YOUR_USERNAME.github.io** directory.
3. Input **git add .** to stage the changes.
4. Input **git commit -m “commit message”** to commit the changes.
5. Input **git push** to push the changes to the remote repository.
6. Repeat from the start for all new changes made.